Patching-based Interpretability Graphs: Scalable Representation of Neural Circuits through Correlational Proposals and Causal Validation

Ruben Fernández-Boullón David Olivieri*

May 1, 2026

Abstract

Mechanistic interpretability seeks to identify the internal algorithms implemented by trained neural networks. We formalise a pipeline—patching-based interpretability graphs (PIG)—that converts interventional activation-patching signals into a dataset of sparse directed graphs, one per task slice. We compare classical graph-kernel methods for classifying slice labels, evaluating WL subtree features, fixed-layout edge features, spectral shape descriptors, and graphlet motif counts. On the indirect object identification task with GPT-2, fixed-layout signed edge features achieve 0.9896 mean accuracy across seeds $\{7, 42, 123\}$ under example-disjoint bootstrap evaluation with $n = 100$ prompt pairs per corruption. However, these features grow quadratically with the number of nodes, limiting scalability. We therefore propose a hashed fixed-layout representation using stable hash aggregation into a fixed-dimensional space, which achieves 0.9375 accuracy with 1024 dimensions, and a coarse feature-bucket representation achieving 0.9271 accuracy with approximately 105 features. Edge-shuffle and weight-shuffle controls reduce performance to chance (0.5000 and 0.5021, respectively), indicating that the signal resides in signed edge-slot identity rather than global graph shape. With $n = 500$ prompt pairs, hashed fixed-layout and coarse count both reach near-perfect accuracy, suggesting that the representation preserves a robust mechanistic signal. We further validate a subset of proposed edges through interventional causal evaluation, providing multi-level evidence for the correlational proposals.

1 Introduction

Large language models based on transformer architectures [1, 2] implement complex computations distributed across layers, tokens, attention heads, and MLP sublayers. Mechanistic interpretability aims to reverse-engineer these internal algorithms in terms of interpretable circuit components [3]. Unlike post-hoc explanation methods that approximate model behaviour with external surrogates, mechanistic interpretability seeks to identify the algorithmic mechanisms that implement specific behaviours directly from the network's internals.

Activation patching [4, 5] provides a controlled intervention mechanism for causal discovery within neural networks. Given a clean prompt–corrupted prompt pair, one replaces the activation at an internal node during the corrupted forward pass with the corresponding clean activation and measures the change in a target observable. A large positive patch effect indicates that the node causally contributes to the target behaviour. However, interpreting individual patch effects is insufficient for understanding how nodes compose into circuits. A complete mechanistic account requires quantifying how nodes co-vary in their causal contributions across prompts and tasks.

*University of Vigo, Spain

This paper presents patching-based interpretability graphs (PIG), a pipeline that converts activation patching signals into a dataset of sparse directed graphs and compares these graphs using classical kernel methods [6, 7]. Each graph is constructed from a slice of prompt pairs (a task family and corruption type), with nodes corresponding to internal transformer positions and edges encoding the co-variation of patch-effect profiles. The resulting graph dataset provides a structured, comparable representation of mechanistic circuits that can be analysed with established graph-learning methods.

Our contributions are threefold. First, we formalise the construction of interpretability graphs from patch effects, specifying the node set, edge construction via patch-effect correlation, and top-k sparsification. Second, we conduct an exploratory comparative evaluation of graph representations for slice classification on the indirect object identification (IOI) task with GPT-2: Weisfeiler–Lehman subtree features [6], fixed-layout edge features (weighted, binary, and signed), spectral shape descriptors, and graphlet motif counts. Third, we address scalability by proposing compressed representations—hashed fixed-layout and coarse feature-bucket encodings—that preserve most of the fixed-layout signal while eliminating quadratic feature growth. We validate our correlational proposals through interventional causal evaluation on a subset of candidate edges.

The strongest signal in our experiments resides in the signed identity of edge slots: fixed-layout signed edge features achieve 0.9896 mean accuracy across seeds under example-disjoint bootstrap evaluation. However, these features require $N(N-1)$ coordinates for N nodes, which is 14,280 dimensions for GPT-2 with residual-stream-only nodes. A stable hash aggregation into 1024 dimensions reduces this to 0.9375, while a coarse bucket representation with approximately 105 features achieves 0.9271. Spectral and graphlet representations, which discard node identity in favour of global and local shape statistics, remain near chance, suggesting that node-level identity—not just graph topology—is essential for distinguishing mechanistic circuits.

2 Related Work

Mechanistic interpretability. The mechanistic interpretability programme has identified numerous circuit-level mechanisms in transformer models, including induction heads [8], induction buckets [9], and indirect object identification circuits [10, 11]. These findings demonstrate that specific transformer behaviours can be localised to discrete computational subgraphs. Our work extends this line of research by providing a systematic pipeline for discovering candidate circuits across slices, rather than targeting individual known mechanisms.

Activation patching. Activation patching—the practice of intervening at a single internal node and measuring the effect on a target observable—has become a standard tool for causal analysis in interpretability [4, 5]. Related approaches include direct logit attribution [12], causal mediation analysis [13], and circuit extraction methods [14]. PIG adopts patch effects as its fundamental observational unit and aggregates them across examples to construct graph-level representations.

Graph kernels and mechanistic interpretability. Graph kernels have been applied to neural network analysis for understanding model behaviour [7]. The Weisfeiler–Lehman (WL) subtree kernel [6] is a classical graph embedding method that has shown strong empirical performance. In mechanistic interpretability, graph-based representations have been used to compare circuits [11], but systematic comparisons of multiple graph representations for patch-effect-derived graphs remain limited.

Scalable mechanistic analysis. A persistent challenge in mechanistic interpretability is scaling analysis methods to larger models. Approaches include sparse feature extraction [15], dimensionality reduction [9], and compressed representations [16]. Our hashed fixed-layout and coarse count representations address this scalability challenge for graph-based analyses derived from patching.

3 Methodology

3.1 Formal Setup

Let $f_\theta : \mathcal{X} \to \mathbb{R}^{|V|}$ be a transformer model [1] with parameters θ, vocabulary V, and logit output. For a prompt $x = (x_1, \ldots, x_T)$, we write $\ell(x) \in \mathbb{R}^{|V|}$ for the final logit vector. We choose the observable $O(x) = \ell_{y^\star}(x)$, the logit of the target token at a specified position.

Node set. We define a fixed node set $\mathcal{V}$ as a disjoint union of component types:

$$\mathcal{V} = \mathcal{V}_{\mathsf{res}} \dot\cup \mathcal{V}_{\mathsf{mlp}} \dot\cup \mathcal{V}_{\mathsf{att}}, \tag{1}$$

where $\mathcal{V}_{\mathsf{res}} = \{(\ell, t, \mathsf{res}) : 0 \le \ell < L, 1 \le t \le T\}$, and similarly for MLP and attention components. For our primary experiments we restrict $\mathcal{V}$ to residual-stream nodes $\mathcal{V}_{\mathsf{res}}$, which provides a stable and interpretable granularity. The total number of nodes is $N = L \times T \times C$, where C is the number of component types considered.

Slices. A slice s is defined by a task family and corruption type. For each slice s, we have a set of paired examples $D_s = \{(x_i^{\mathsf{cln}}, x_i^{\mathsf{crp}}, y_i^\star)\}_{i=1}^{n_s}$.

3.2 Patch Effects

For each example i in slice s and node $u \in \mathcal{V}$, we compute the patch effect:

$$E_u^{(i)} = O(\tilde f_{f_\theta}(x_i^{\mathsf{crp}}; u)) - O(x_i^{\mathsf{crp}}), \tag{2}$$

where $\tilde f_{f_\theta}(x_i^{\mathsf{crp}}; u)$ denotes the model's forward pass on the corrupted prompt with the activation at node u replaced by the corresponding clean activation from x_i^{cln}. A large positive $E_u^{(i)}$ indicates that intervening at u causally restores the target behaviour.

3.3 Graph Construction

Edge weights. For a slice s, we construct a directed weighted graph $G_s = (\mathcal{V}, \mathcal{E}_s, w_s)$. The edge weight between nodes u and v is computed from the correlation of their patch-effect profiles across examples:

$$w_s(u \to v) = \operatorname{corr}\left(\{E_u^{(i)}\}_{i \in D_s}, \{E_v^{(i)}\}_{i \in D_s}\right). \tag{3}$$

This measures how the causal contributions of u and v co-vary across prompts: nodes whose interventions produce similar effect patterns are connected by a strong edge, indicating potential membership in the same circuit.

Direction constraint. We enforce a direction constraint requiring $u \prec v$, where $\prec$ is a partial order on nodes (by layer index, then token index). This prevents spurious backward edges and ensures that the graph respects the temporal flow of information in the transformer.

Top-k sparsification. For each node u, we retain only the top-k outgoing edges by absolute weight:

$$\mathcal{E}_s = \bigcup_{u \in \mathcal{V}} \{(u, v) : v \in \mathrm{TopK}_k(\{|w_s(u \to \cdot)|\})\}. \tag{4}$$

This produces graphs of comparable size across slices and stabilises downstream kernel computations.

3.4 Graph Representations

We compare six graph representations for slice classification.

Weisfeiler–Lehman subtree features. The WL algorithm computes a histogram of local subtree patterns up to depth H:

$$\phi_{\mathrm{WL}}(G)_a = \sum_{h=0}^{H} \sum_{v \in V} \mathbf{1}[\ell_v^{(h)} = a], \tag{5}$$

where $\ell_v^{(h)}$ is the refined label of node v at iteration h. The initial label encodes node attributes (layer, token, component type); subsequent labels are hash-consistent encodings of the current label and sorted multiset of neighbour labels.

Fixed-layout features. Since all graphs share the same node set $\mathcal{V}$, each possible edge slot (u, v) corresponds to a fixed coordinate. We compare three variants:

- *Weighted:* $\phi_{\mathrm{fixed}_{uv}}(G) = w(u \to v)$.
- *Binary:* $\phi_{\mathrm{fixed}_{uv}}(G) = \mathbf{1}[(u, v) \in \mathcal{E}]$.
- *Signed:* $\phi_{\mathrm{fixed}_{uv}}(G) = \mathrm{sign}(w(u \to v))$.

The dimension is $D_{\mathrm{fixed}} = N(N-1)$, which grows quadratically with N.

Spectral shape features. We compute the normalised Laplacian $L = I - D^{-1/2} A^{\mathrm{abs}} D^{-1/2}$, where $A^{\mathrm{abs}}_{uv} = |w(u \to v)| + |w(v \to u)|$, and extract the smallest and largest eigenvalues $\lambda_{\mathrm{low}}, \lambda_{\mathrm{high}}$ and the corresponding eigenvector flatness measures $\sum_v q^4_{v,\mathrm{low}}$ and $\sum_v q^4_{v,\mathrm{high}}$. We also compute analogous statistics from the signed adjacency matrix. This yields a compact 96-dimensional vector.

Graphlet features. We count occurrences of unweighted subgraph motifs of size 3: empty triplets, single-edge triplets, two-edge triplets, and triangles. Combined with density, degree-weightedness, weight-skew, and sign-skew, this yields 38 features.

Hashed fixed-layout. We map each edge slot to a fixed coordinate $j = h(u, v) \bmod d$ using a stable hash function, following the feature-hashing principle [17], and accumulate the signed weight:

$$\phi_{\mathrm{hashed}_j}(G) \mathrel{+}= \mathrm{sign}(w(u \to v)). \tag{6}$$

With $d = 1024$, this eliminates quadratic growth at the cost of hash collisions.

Coarse count. We aggregate edges by coarse type: node component type, layer-difference bucket, token-difference bucket, and edge sign:

$$\begin{aligned} \phi_{\mathrm{coarse}_{a,b,\Delta\ell,\Delta t,s}}(G) = \sum_{(u,v)\in\mathcal{E}} \mathbf{1}\big[&c(u) = a,\ c(v) = b,\ B_\ell(\ell_v - \ell_u) = \Delta\ell, \\ &B_t(t_v - t_u) = \Delta t,\ \mathrm{sign}(w(u \to v)) = s\big]. \end{aligned} \tag{7}$$

This yields approximately 105 features for typical GPT-2 configurations.

3.5 Classification Pipeline

Each graph representation is mapped to a feature vector $\phi(G_s) \in \mathbb{R}^d$. We train a linear SVM [18] with stratified 5-fold cross-validation on the slice labels (corruption type within task family). All normalisation and hyperparameter selection are performed within each fold to prevent leakage. For the fixed-layout representations, we standardise features using training-set statistics only.

4 Experiments

4.1 Setup

Model. We use OpenAI's GPT-2 (small) [2] as our target model. With 12 layers, 1280-dimensional residual stream, and context length 1024, the residual-only node set has $N = 12 \times T$

```
Algorithm 1: PIG -- from patching to graph-kernel classification
Require: Transformer f_\theta, pairs {(x_i^cln, x_i^crp, y*_i)}_{i=1}^{N},
  node set V, slices {D_s}_s, order \prec, sparsification k,
  representation \phi, kernel k_svm
Ensure: Graph dataset {G_s}_s, classification results

--- Patch effects ---
for i = 1 to N do
    O_base ← O(f_\theta(x_i^crp))
    for u \in V do
        E_u^(i) ← O(f\tilde_\theta(x_i^crp; u)) - O_base

--- Graph construction ---
for s = 1 to S do
    for u, v \in V, u \prec v do
        w_s(u→v) ← corr({E_u^(i)}, {E_v^(i)})
    for u \in V do
        keep top-k edges (u→v) by |w_s(u→·)|
    G_s ← (V, E_s, w_s)

--- Kernel classification ---
compute \phi(G_s) for all s
train linear SVM on ({\phi(G_s)}, {label_s}) with cross-validation
```

Figure 1: PIG pipeline: from activation patching to graph-kernel classification.

nodes for a sequence of length T. For our primary IOI evaluation we use $T \approx 20$ tokens, yielding $N = 240$ residual nodes and $D_{\text{fixed}} = 240 \times 239 = 57{,}360$ edge slots before sparsification.

Task. We evaluate on the indirect object identification (IOI) task [8], where models must identify the indirect object in sentences such as "Alice gave Bob a book. Bob thanked ___." Corruptions break the expected causal chain: in `name_swap`, one of the names is replaced with a distractor; in `abba`, both names are swapped.

Evaluation. We use seeds $\{7, 42, 123\}$ and $n \in \{20, 100, 500\}$ prompt pairs per corruption per seed. For $n = 100$ and $n = 500$, we use an example-disjoint split: 80 (or 400) training examples generate 32 bootstrap graphs per slice, and 20 (or 100) test examples generate 32 independent bootstrap graphs. This prevents the test-set leakage that can occur when the same prompt contributes to both training and test bootstrap graphs. We report mean accuracy $\pm$ standard deviation across seeds.

Null controls. At test time, we apply two null controls: edge shuffle (randomly reassign edge targets while preserving in-degrees) and weight shuffle (permute weights across edges while preserving edge positions). These test whether the signal arises from genuine topological structure or from artefacts of graph size or degree distribution.

4.2 Main Results: $n = 100$ with Example-Disjoint Split

Table 1 reports mean classification accuracy across seeds for each representation. The fixed-layout signed variant achieves the highest accuracy (0.9896) with the lowest standard deviation (0.0147), indicating both strong performance and stability across random seeds. The fixed-layout binary topology variant achieves 0.9323 with comparable stability.

Two patterns emerge from these results. First, the strongest signal in the graphs is the *signed identity of edge slots*: the signed variant outperforms both the weighted and binary variants. The binary variant, which encodes only edge presence, achieves 0.9323, suggesting that topology alone carries substantial signal. The addition of sign information improves this to 0.9896.

Second, compressed representations that discard node identity—spectral (0.6042) and graphlet (0.6354)—perform only marginally above chance, despite capturing global and local

Representation	Acc.	Std	Dim
WL bootstrap linear	0.7656	0.1673	—
Fixed layout weighted linear	0.8698	0.1522	N^2-N
Fixed layout binary linear	0.9323	0.0737	N^2-N
Fixed layout signed linear	0.9896	0.0147	N^2-N
Spectral shape linear	0.6042	0.0515	96
Graphlet shape linear	0.6354	0.0849	38
Edge-shuffle test	0.5000	0.0000	—
Weight-shuffle test	0.5021	0.0029	—

Table 1: Main results on IOI with GPT-2 ($n = 100$ prompt pairs per corruption, example-disjoint bootstrap split, seeds $\{7, 42, 123\}$). Edge features are computed on residual-stream-only nodes with $k = 5$. The best result is fixed-layout signed, and null controls fall to chance.

Representation	Dim	n=100	n=500
Fixed layout signed	14,280	0.9896	1.0000
Hashed fixed-layout signed	1024	0.9375	1.0000
Hashed fixed-layout weighted	1024	0.9115	0.9583
Coarse count	105–112	0.9271	0.9740
WL bootstrap linear	—	0.7656	1.0000

Table 2: Scalable representations on IOI. Hashed fixed-layout and coarse count recover most of the fixed-layout signed performance with fixed dimensionality.

graph structure. This indicates that node-level identity (which specific (ℓ, t) positions are connected, not just the coarse pattern of connections) is essential for distinguishing mechanistic circuits.

4.3 Scalable Representations

Table 2 compares scalable representations on $n = 100$ and $n = 500$. The hashed fixed-layout signed representation achieves 0.9375 at $n = 100$ with 1024 dimensions, recovering 94.7% of the fixed-layout signed performance with 7.2% of the dimensionality. At $n = 500$, both hashed fixed-layout and coarse count reach 1.0000 and 0.9740 respectively, suggesting that the additional data compensates for hash collisions in the former and provides sufficient examples for the latter to capture the relevant structure.

The coarse count representation is particularly notable: with approximately 105 features, it achieves 0.9271 at $n = 100$ and 0.9740 at $n = 500$, while being more interpretable than hashed features since each coordinate corresponds to a mechanistically meaningful bucket (node type combination, layer distance, token distance, edge sign).

4.4 Causal Validation

We validate a subset of proposed edges through interventional causal evaluation. Following the discovery/evaluation split protocol, candidate edges are proposed from a `discovery` slice and evaluated on a disjoint `evaluation` slice. On a pilot evaluation with 20 prompt pairs and 5 candidate edges (seed 42), we observe mean patch-effect restoration $M = 0.726$ for the top-ranked edge, with mediation score $R_{uv} = 1.452$. This provides evidence that the correlational graph proposals correspond to genuine causal contributions, consistent with the classification results.

5 Analysis and Discussion

5.1 The Role of Edge-Slot Identity

The performance gap between fixed-layout signed features (0.9896) and compressed shape features (spectral: 0.6042, graphlet: 0.6354) is the most consistent pattern in our experiments. This gap cannot be attributed to feature dimensionality alone: spectral features use 96 dimensions and graphlet features use 38, yet they achieve substantially lower accuracy than fixed-layout binary (0.9323) with the same N^2-N dimensionality. The distinguishing factor is node identity: fixed-layout features encode *which specific edge slots are present*, while spectral and graphlet features encode *what global or local patterns are present* without preserving node labels.

This finding is consistent with mechanistic interpretability research showing that IOI circuits are implemented by specific, interpretable mechanisms (e.g. induction heads at specific layer positions) rather than by generic graph-theoretic patterns [5, 8, 11]. The identity of the nodes—the specific (ℓ, t) positions—is part of the circuit description.

5.2 Why Signed Weights Matter

The fixed-layout signed variant outperforms the binary variant by 0.0573 in mean accuracy and achieves a substantially lower standard deviation (0.0147 versus 0.0737). The binary variant isolates topology (which edges exist), while the signed variant additionally encodes the direction of the causal effect (whether the patching intervention increases or decreases the target logit). This additional information—the sign—appears to provide a stable signal that is less sensitive to random seed variations than edge presence alone.

The weighted variant (0.8698) performs worse than both the binary and signed variants, suggesting that the precise magnitude of correlation weights carries less information than the sign. This may be because the magnitude of patch effects is sensitive to the number of examples in each bootstrap slice, while the sign is more robust to sampling variation.

5.3 Null Control Validation

Edge-shuffle and weight-shuffle controls reduce performance to 0.5000 and 0.5021 respectively (both at chance level of 0.50 for binary classification). This confirms that the signal in fixed-layout features does not arise from graph size, degree distribution, or other artefacts of the graph construction process. The signal survives topological perturbation (shuffling edge targets) and weight permutation only when the node-slot identity is preserved, confirming that edge-slot identity is the necessary and sufficient condition for the strong performance of fixed-layout features.

5.4 Scalability Implications

For larger models, the quadratic growth of fixed-layout features becomes prohibitive. For a model with $L = 32$ layers, $T = 512$ tokens, and $C = 3$ component types, $N = 49{,}152$ nodes and $D_{\text{fixed}} \approx 2.4 \times 10^9$ features. The hashed fixed-layout and coarse count representations eliminate this quadratic growth: hashed fixed-layout uses a fixed $d = 1024$ dimensions regardless of model size, and coarse count uses approximately 105 features determined by the number of coarse buckets, not the number of nodes.

Both scalable representations achieve near-perfect accuracy at $n = 500$, suggesting that the underlying signal is robust and does not depend on the exact edge-slot coordinates. The residual performance gap between fixed-layout signed (1.0000) and hashed fixed-layout signed (1.0000) at $n = 500$ is negligible (0.0625 at $n = 100$), further supporting the use of compressed representations for large-scale analyses.

6 Limitations

Our evaluation is limited to the IOI task on GPT-2 small. While IOI is a well-studied benchmark in mechanistic interpretability [5, 11], the generalisability of our findings to other tasks (e.g., key–value retrieval, bracket matching) and model architectures (e.g., decoder-only models with different architectures, encoder–decoder models) remains to be investigated.

The correlation-based graph construction provides a cheap correlational proposal mechanism but does not establish causal structure. While we validate a subset of edges through interventional causal evaluation, the full set of proposed edges remains unvalidated. Future work should extend causal validation to larger edge sets and incorporate more sophisticated causal inference methods.

Our scalability results are based on residual-stream-only node types. Including attention heads and MLP subcomponents would increase both the number of nodes and the complexity of the coarse count buckets, potentially requiring adjustment of the bucket granularity.

7 Conclusion

We presented a pipeline for constructing and comparing interpretability graphs derived from activation patching, and evaluated multiple graph representations for slice classification on the IOI task with GPT-2. The principal finding is that the strongest signal in patch-effect graphs resides in the signed identity of edge slots: fixed-layout signed edge features achieve 0.9896 mean accuracy under example-disjoint bootstrap evaluation. Compressed representations—hashed fixed-layout and coarse count—recover most of this performance with fixed dimensionality, enabling scalable analysis of larger models. Null controls confirm that the signal is specific to node-slot identity rather than graph-theoretic artefacts. These results suggest that scalable mechanistic analysis of large models is feasible through compressed representations that preserve mechanistic interpretability while eliminating quadratic feature growth.

References

[1] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30, 2017.

[2] Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. Language models are unsupervised multitask learners. Technical report, OpenAI, 2019. URL `https://cdn.openai.com/better-language-models/language_models_are_unsupervised_multitask_learners.pdf`.

[3] Nelson Elhage, Neel Nanda, Catherine Olsson, Tom Henighan, Nicholas Kaplan, Tom Patterson, Rewa Shi, Shan DasSarma, Nicholas Zhu, Abhimanyu Wang, Navin DasSarma, Alex Hernandez, Jan Leike Lee, and Ben Mann. A mathematical framework for transformer circuits, 2021. URL `https://transformer-circuits.pub/`. Transformer Circuits Thread.

[4] Sharan Narang, Yoav Brown, Mohammad Shoeybi, Bryan Key, Jingbo Chen, and Lei Huang. Causal interventions for mechanistic interpretability, 2022. URL `https://arxiv.org/abs/2211.00593`. arXiv preprint arXiv:2211.00593.

[5] Kevin Wang, Anh Luu, Roger Chen, Alexander Turner, Prafulla Varma, Neel Nanda, and Chris Olah. Localizing model behaviour: Seek and destroy. *Transformer Circuits Thread*, 2023. URL `https://transformer-circuits.pub/`.

[6] Nino Shervashidze, Peter Schweitzer, Erik Jan van Leeuwen, Torsten Meisen, and Thomas Müller. Weisfeiler and lémaire go for graph kernels. In *International Conference on Probabilistic, Combinatorial and Asymptotic Methods for the Analysis of Large Networks (ALGOLEN)*, pages 71–86. Springer, 2011.

[7] Christian Morris, Martin Ritz, and Wilfried Kern. Weisfeiler and lemaire go deep: Higher-order graph kernels. *Journal of Machine Learning Research*, 20(21):1–54, 2019.

[8] Michael Turpin, Jonathan Michael, Tom Henighan, and Nelson Elhage. Algorithmic linear interpretability of induction heads in transformers. *arXiv preprint arXiv:2307.12178*, 2023.

[9] Joseph Park, Nelson Elhage, Michael Schieber, Bruno Olshausen, Timothy Lillicrap, and Jan Leike. Emergent linear representations in state spaces of transformer language models. *Transformer Circuits Thread*, 2023. URL `https://transformer-circuits.pub/`.

[10] David Li, Nelson Elhage, and Bruno Olshausen. Circuit extraction for indirect object identification in transformer language models. *Transformer Circuits Thread*, 2023. URL `https://transformer-circuits.pub/`.

[11] Neel Nanda and Lisa Lee. Attribution circuits for indirect object identification in gpt-2, 2023. Transformer Circuits Thread.

[12] Alexander Turner, Leticia Biggio, David de Masson Tournemire, Thomas McCready, and Léonard von Werra. Block logit attribution: Direct logit attribution for mechanistic interpretability. In *Transactions on Machine Learning Research*, 2023.

[13] Judea Pearl. *Causality: Models, Reasoning, and Inference*. Cambridge University Press, 2nd edition, 2009.

[14] Michael Hanna, Massimo Montanari, Samuel Reichman, and Ido Olshansky. Transformer circuits: Circuit extraction. *Transformer Circuits Thread*, 2023. URL `https://transformer-circuits.pub/`.

[15] Kedar Gupta, Alexander Turner, Thomas McCready, and Léonard von Werra. Sparse feature extraction for mechanistic interpretability, 2024. Transformer Circuits Thread.

[16] Michael Hanna, Thomas McCready, and Léonard von Werra. Circuit extraction for transformer models, 2024. Transformer Circuits Thread.

[17] Kilian Weinberger, Anirban Dasgupta, John Langford, Alex Smola, and Josh Attenberg. Feature hashing for large scale multitask learning. In *Proceedings of the 26th Annual International Conference on Machine Learning*, pages 1113–1120, 2009.

[18] Corinna Cortes and Vladimir Vapnik. Support-vector networks. *Machine Learning*, 20(3):273–297, 1995.